COURSE PROJECT

Building a Compiler and a Virtual Machine

A small C-like language taken through every classical phase in C++17: from source text, to tokens, to an AST, to intermediate code, to machine code, and finally executed on our own virtual machine.

C++17 | Lexer + Parser | IR & Code Generation | Virtual Machine | Release + Debug builds

OVERVIEW

What the project is

It is a complete toolchain for a tiny C subset. You write a program, the compiler translates it down to instructions, and our virtual machine runs those instructions and gives back a result.

- **Front end** — reads the source and understands its structure (Lexer, Parser).
- **Middle end** — turns the structure into simple intermediate code (IRGen, LogicalCode).
- **Back end** — produces binary instructions and an executable file (CodeGenIR, Optimizer, Assembler, ExecIO).
- **Runtime** — loads and executes the program (Loader, Virtual Machine, Debugger).

ARCHITECTURE

The compilation pipeline

Each phase has one job and hands its result to the next. This is the same staged design used by real compilers.

1
Lexer
source text → tokens

2
Parser
tokens → AST

3
IRGen
AST → three-address IR

4
LogicalCode
lower && || ! to branches

5
CodeGenIR
IR → binary instructions

6
Optimizer
peephole cleanup

7
Linker
resolve symbols & relocs

8
Assembler
readable disassembly

9
ExecIO
write .exec file

10
Loader + VM
execute the program

PHASE 1

Lexer — text into tokens

The lexer scans the source one character at a time and groups characters into meaningful tokens. It also skips whitespace and comments and tracks line numbers for error messages.

Recognizes

- Numbers, strings, identifiers
- Keywords: `if while for return ...`
- Operators incl. `== != <= && ||`

Input → Output

```
int a = 5;
// becomes the token stream:
INT_KW  VARIABLE(a)  ASSIGN
NUMBER(5)  SEMICOLON
```

`include/Lexer.h` · `include/Common.h`

PHASE 2

Parser — tokens into an AST

A recursive-descent parser checks that the tokens form valid grammar and builds an Abstract Syntax Tree. Operator precedence is handled by a chain of functions, one per precedence level.

Handles

- Declarations: functions, variables
- Statements: `if/while/for/switch`
- Expressions by precedence

AST for `a + b`

```
   BINOP "+"
   /      \
VAR_REF   VAR_REF
  (a)       (b)
```

`include/Parser.h`

PHASES 3 & 4

Intermediate code

The AST is flattened into simple three-address instructions that are easy to translate. A separate pass rewrites the logical operators into compares and branches.

IRGen: AST → IR

```
// c = a + b
t0 = a + b
c  = t0
```

LogicalCode

Turns `&&`, `||`, `!` into branch + compare IR, so the back end only deals with simple jumps.

`src/IRGen.cpp` · `src/LogicalCode.cpp` · `include/ir.h`

PHASES 5 - 8

Back end — producing machine code

The IR is lowered into the virtual machine's binary instruction set, cleaned up, disassembled for inspection, and written to an executable file.

- **CodeGenIR** assigns registers and emits fixed 4-byte instructions, a constant pool, symbols, relocations, and jump tables.
- **Optimizer** runs a peephole pass (removes dead consecutive writes).
- **Assembler** prints a readable listing of the instructions.
- **ExecIO** writes a self-describing `.exec` file with a header and sections (code, data, symbols, relocs, jump table).

LINKER

Linker — resolving symbols

When the code generator emits a function call, the target address may not be known yet. It leaves a placeholder and records a relocation; the linker fills in the real address.

Inputs

- **Symbols**: function name + address
- **Relocations**: call site + target name

What it does

```
for each relocation r:
  addr = symbols[r.name]
  code[r.site].rs1 = addr
```

Reports any unresolved reference.

`include/linker.h` · `src/linker.cpp`

RUNTIME

The Virtual Machine

A register-based virtual machine that runs our instructions using the classic fetch → decode → execute cycle.

Architecture

- 32 registers, `pc`, `sp`, `bp`, flags
- 1 MB memory: Code / Data / Heap / Stack
- 16 / 32 / 64-bit word sizes

Instruction set

- Arithmetic, compare, branch, jump
- Call / return with stack frames
- Push/pop, load/store, heap alloc, jump tables

`include/vm.h` · `src/vm.cpp`

ENGINEERING

Two standard builds

The project builds in two configurations through both a Makefile and CMake, the way real C++ projects are organized.

Release

```
make
# -O2 -DNDEBUG
./build/release/compiler
```

Optimized, for normal runs.

Debug

```
make debug
# -O0 -g3 -DDEBUG_BUILD
# ASan + UBSan, gdb symbols
./build/debug/compiler
```

Prints a per-instruction VM trace.

DEBUG BUILD IN ACTION

Watching the VM execute

In the debug build, every instruction the VM fetches is traced to the screen, so the fetch-decode-execute loop is visible step by step.

```
[trace] pc=0    PUSH_BP     rd=0   rs1=0   rs2=0
[trace] pc=1    SET_BP_SP   rd=0   rs1=0   rs2=0
[trace] pc=2    MOV_CONST   rd=1   rs1=0   rs2=0
[trace] pc=3    ADD         rd=2   rs1=1   rs2=0
...
[trace] pc=10   ADD         rd=8   rs1=2   rs2=4
[trace] pc=12   ADD         rd=0   rs1=6   rs2=0
Result R0 = 15 (expected 15)
```

END TO END

A full example

This demo program travels through every phase and the VM returns `15`.

Source

```
int main() {
  int a = 5;
  int b = 10;
  int c = 0;
  if (a < b) {
    c = a + b;
  }
  return c;
}
```

Journey

- Lexer → tokens
- Parser → AST
- IRGen + LogicalCode → IR
- CodeGenIR → instructions
- VM executes → **R0 = 15**

SUMMARY

What it demonstrates

- A complete **front end**: lexer and recursive-descent parser.
- A **middle end** with intermediate code and a logical-lowering pass.
- A **back end** that emits binary code and a real executable file format.
- A working **register-based virtual machine** with fetch-decode-execute.
- Standard engineering: **release and debug builds**, warnings clean, sanitizers pass.

> **Result:** the demo program compiles and runs correctly, returning 15 across all run modes (default, `--exec`, `--debug`).

THANK YOU

Questions?

Source layout: `include/` headers, `src/` translation units, `CMakeLists.txt` and `Makefile` for building.

Lexer · Parser · IRGen · LogicalCode · CodeGenIR · Optimizer · Linker · Assembler · ExecIO · Loader · Virtual Machine